SCCM Application Migration to Intune Win32 (Repackaging)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — SCCM Application Migration to Intune Win32 (Repackaging) 4

What you get . . . 4

What this delivers . . . 4

The Phoenixtekk deployment model . . . 4

Outcome . . . 4

1. Solution Overview — SCCM Application Migration to Intune Win32 (Repackaging) 5

What this migration is . . . 5

Why this is its own project (not net-new packaging) . . . 5

How it works . . . 5

What maps to what (SCCM → Intune Win32) . . . 5

What you can control . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard Phoenixtekk deployment model . . . 6

2. Requirements Guide — SCCM Application Migration to Intune Win32 7

2.1 Intune licensing and admin rights (target) . . . 7

2.2 Configuration Manager read access (source) . . . 7

2.3 Win32 Content Prep Tool . . . 7

2.4 Source installers / content . . . 7

2.5 Detection method captured per app . . . 7

2.6 Dependency and supersedence map . . . 8

2.7 Packaging workstation . . . 8

2.8 Pilot ring group and coexistence/rollback plan . . . 8

Checklist . . . 8

3. Implementation Guide — SCCM Application Migration to Intune Win32 8

Named objects (reuse these names verbatim everywhere) . . . 9

Step 1 — Inventory the SCCM application estate . . . 9

Step 2 — Prioritize into migration waves . . . 9

Step 3 — Repackage the app into a `.intunewin` . . . 10

Step 4 — Create the ring groups . . . 10

Step 5 — (If needed) Repackage and create the dependency app first . . . 10

Step 6 — Create the migrated Win32 app . . . 10

Step 7 — Set install/uninstall commands and behavior (carry over from SCCM) . . . 10

Step 8 — Author requirement rules to match SCCM . . . 11

Step 9 — Author detection rules to match SCCM (parity) . . . 11

Step 10 — Declare dependencies and supersedence to match SCCM . . . 11
Step 11 — Assign to the pilot ring (start coexistence) . . . 12
Step 12 — Apply and validate on the pilot (per-app gate) . . . 12
Step 13 — Retire the SCCM deployment for the pilot, then expand . . . 12
5. Testing Guide — SCCM Application Migration to Intune Win32 13
Test 1 — Repackage produced a valid `.intunewin` . . . 13
Test 2 — Install/uninstall parity . . . 13
Test 3 — Detection parity . . . 13
Test 4 — Requirements applied . . . 13
Test 5 — Dependencies and supersedence . . . 13
Test 6 — Assignment (ring) parity . . . 13
Test 7 — Coverage and deployment-success report . . . 13
Pass/fail gate . . . 14
6. Troubleshooting Guide — SCCM Application Migration to Intune Win32 14
A. App installs but Intune reports Failed / not detected . . . 14
B. Install fails on the device . . . 14
C. App offered to the wrong devices . . . 14
D. Supersedence / upgrade problems . . . 15
E. Both SCCM and Intune fighting over the app . . . 15
F. Rollback . . . 15
G. Diagnostics quick reference . . . 15
H. When to escalate . . . 15
7. Best Practices — SCCM Application Migration to Intune Win32 16
8. Deliverables Summary — SCCM Application Migration to Intune Win32 16
Documentation . . . 16
PowerShell scripts . . . 17
What the buyer receives . . . 17
Outcome . . . 17
Appendix A — PowerShell Scripts 19

Executive Summary — SCCM Application Migration to Intune Win32 (Repackaging)

Phoenixtekk Workplace Marketplace · Migrations

Moving software delivery off Configuration Manager means every SCCM application — with its detection method, requirements, dependencies, and deployment types — has to be repackaged as an Intune Win32 app and validated so nobody loses software mid-migration. Done by hand it is slow, inconsistent, and easy to get wrong: a mismatched detection rule alone makes Intune report an app as failed when it actually installed.

This is a complete, deploy-it-today package for migrating the SCCM application estate to Intune Win32, written for an administrator carrying out the move without prior repackaging experience.

What you get

• A plain-language Solution Overview of how SCCM applications map to Intune Win32 apps, the repackage-and-validate model, SCCM/Intune coexistence, and the ringed cutover.

• A Requirements Guide with verify / implement-if-missing for every prerequisite.

• An Implementation Guide of precise, ordered steps on both consoles — inventory SCCM apps, repackage each into `.intunewin`, recreate detection/requirement/dependency/supersedence parity, assign by ring, run coexistence, validate, cut over, and retire SCCM.

• Five PowerShell scripts for SCCM app inventory, install/detection parity validation, repackage coverage + deployment-success reporting, safe remediation, and the app-assignment ring builder.

• Testing and Troubleshooting guides with explicit per-app pass/fail gates.

What this delivers

A repeatable inventory-repackage-validate loop: every SCCM application becomes an Intune Win32 app with matching detection, requirements, dependencies, and assignments; SCCM keeps deploying during coexistence until each app is proven; and the SCCM deployment is retired ring by ring only after Intune is validated.

The Phoenixtekk deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Your SCCM application estate repackaged into Intune Win32 and proven to deploy — correct detection, requirements, and assignments, validated before cutover with SCCM coexistence — so no user loses software during the move, proven on a pilot before you scale.

1. Solution Overview — SCCM Application Migration to Intune Win32 (Repackaging)

What this migration is

Configuration Manager (SCCM/MECM) delivers software as `Application` objects, each with one or more `deployment types` (MSI, script/EXE, App-V, etc.), a `detection method`, `requirement rules`, `dependencies`, and (optionally) `supersedence`. Intune delivers the same software as `Win32 apps` (`.intunewin` packages) with their own detection, requirement, dependency, and supersedence model. This solution is the disciplined migration that takes each SCCM application and repackages it into an Intune Win32 app with parity — so the same software installs on the same devices, detected the same way, without anyone losing an app mid-move.

Why this is its own project (not net-new packaging)

Packaging a brand-new app for Intune is a blank-slate exercise. Migrating an SCCM estate is not: the detection methods, requirements, dependency chains, and supersedence relationships already exist and encode years of decisions. The work here is to inventory what SCCM already does for every app, translate each of those constructs faithfully into the Intune equivalent, run both systems in coexistence while you cut over ring by ring, validate that each app still deploys, and only then retire the SCCM deployment. The value is parity and proof, at estate scale.

How it works

1. Inventory the SCCM application estate — applications, their deployment types, detection methods, requirements, dependencies, and supersedence — with the Configuration Manager module.
1. Prioritize apps into migration waves (start with simple, high-value, low-risk apps).
2. Repackage each app's authoritative deployment type into an `.intunewin` file with the Win32 Content Prep Tool, carrying over the install/uninstall commands.
1. Recreate detection, requirement, dependency, and supersedence rules in Intune to match SCCM.
2. Recreate assignments as Intune ring groups (pilot then broad), mirroring the SCCM collections.
3. Run SCCM/Intune coexistence during transition — SCCM keeps deploying until the Intune app is validated on the same devices — then cut over by ring.
1. Validate install and detection parity per app, expand to broad, and retire the SCCM deployment.

What maps to what (SCCM → Intune Win32)

- `Application` + authoritative `deployment type` → one `Win32 app` (`.intunewin`).
- `Install/uninstall program` (deployment type) → Win32 `Install/uninstall command`.
- `Detection method` (MSI product code / file / registry / script) → Win32 `detection rule`.
- `Requirements` (OS, architecture, disk, custom) → Win32 `requirement rules`.

- `Dependencies` → Win32 `Dependencies` (auto-install the prerequisite app).
- `Supersedence` → Win32 `Supersedence` (uninstall-previous or in-place upgrade).
- `Deployment to a collection` → `assignment` to an Entra ring group (Required or Available).

What you can control

- Which deployment type is authoritative when an SCCM app has several.
- Detection parity — the same product code / file / registry / script logic in Intune.
- Assignment model — Required (mandatory) vs Available (Company Portal), by ring.
- Coexistence duration per app and the order of cutover across rings.
- Supersedence behavior — clean replace vs in-place upgrade — matching the installer.

Benefits

- No lost software — every user keeps every app across the move.
- Detection parity — Intune reports Installed for the same devices SCCM did.
- Repeatable — the same inventory-repackage-validate loop runs for every app.
- Safe — coexistence plus ringed cutover means a bad build never reaches the fleet.
- Auditable — a coverage report proves how much of the estate is migrated and deploying.

Limitations (be honest with buyers)

- App-V, script-heavy, or user-context deployment types may need rework, not a straight repackage; this solution flags them for a packaging decision rather than auto-converting them.
- The Win32 Content Prep Tool wraps content; it does not fix a broken installer or a bad silent switch — the source install must already be reliable.
- Detection must be authored deliberately — Intune's auto-detect is less reliable than a captured MSI product code / file / registry / script rule.
- SCCM and Intune must not both *enforce* the same app on the same device at the same time in a conflicting way; coexistence means SCCM deploys until Intune is validated, then you stop the SCCM deployment for that ring.

The standard Phoenixtekk deployment model

> *Inventory SCCM apps → repackage into `.intunewin` with detection/requirement/dependency parity → assign to a pilot ring → validate install + detection per app → expand to broad and retire the SCCM deployment. Phoenixtekk pattern: Assess → Configure → Pilot → Validate → Expand.*

2. Requirements Guide — SCCM Application Migration to Intune Win32

Verify **every** requirement before repackaging. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites (target tenant, Intune licensing, source access, inventory, enrollment, pilot/waves, change control) are in `global-requirements-migrations.md` and are assumed here.

2.1 Intune licensing and admin rights (target)

Description: An Intune-bearing license plan and the roles to create and assign Win32 apps. Why: You cannot create `.intunewin` apps or assignments in the target without licensing and rights. Verify: `https://intune.microsoft.com` loads; you hold Intune Administrator; go to `Apps` → `Windows`. Implement: Assign M365 E3/E5, Business Premium, or Intune Plan 1; grant least-privilege admin roles.

2.2 Configuration Manager read access (source)

Description: Administrative read access to the Configuration Manager site to inventory applications. Why: The migration begins by exporting each SCCM application's deployment types, detection methods, requirements, dependencies, and supersedence. Verify: The Configuration Manager console opens to `Software Library` → `Application Management` → `Applications`; the Configuration Manager PowerShell module loads; run `SCCMAPP-Discovery.ps1`. Implement: Obtain a Read-only Analyst (or equivalent) SCCM role and install the ConfigMgr console / module (`Microsoft.ConfigurationManagement.PowerShell`) on the admin workstation.

2.3 Win32 Content Prep Tool

Description: `IntuneWinAppUtil.exe`, the Microsoft tool that wraps a source folder into `.intunewin`. Why: Every SCCM app is repackaged into an `.intunewin` file before it can be uploaded to Intune. Verify: `IntuneWinAppUtil.exe` is present on the packaging workstation (e.g. `C:\Tools\IntuneWinAppUtil\`). Implement: Download the Microsoft Win32 Content Prep Tool and place it on the packaging workstation.

2.4 Source installers / content

Description: The original installer media (MSI/EXE + switches) for each in-scope SCCM application. Why: The Content Prep Tool wraps the source; without clean install media you cannot repackage. Verify: For each app, the SCCM deployment type's content location resolves and the silent install command is known; `SCCMAPP-Discovery.ps1` reports the source path and install command. Implement: Recover the content from the SCCM content source share (`Get-CMDeploymentType` reports the content path) or the vendor; confirm the silent switches install cleanly on a test device.

2.5 Detection method captured per app

Description: The authoritative detection logic for each app (MSI product code, file+version, registry value, or a detection script) carried over from SCCM. Why: Intune reports Installed only when the detection rule matches; parity depends on this. Verify: `SCCMAPP-Discovery.ps1` exports each deployment type's

detection method; confirm it is a type Intune supports (MSI / file / registry / script). Implement: Where SCCM used a complex/script detection, port the script or convert it to an equivalent file/registry/MSI rule; record it as the app's detection rule for Step 7 of the implementation.

2.6 Dependency and supersedence map

Description: The dependency chains and supersedence relationships between SCCM apps. Why: Intune must recreate the same prerequisite auto-installs and version-replacement behavior. Verify: `SCCMAPP-Discovery.ps1` lists each app's dependencies and supersedence; review for cycles. Implement: Package/create dependency apps first; record which app supersedes which for Step 8.

2.7 Packaging workstation

Description: A clean Windows workstation to stage source, run the Content Prep Tool, and test installs. Why: Repackaging and per-app install testing happen off the production fleet. Verify: A Windows 10/11 workstation with the Content Prep Tool and space for source + output. Implement: Provision the workstation; create `C:\Packaging\` (source) and `C:\Packaging\Output\`.

2.8 Pilot ring group and coexistence/rollback plan

Description: An Entra security group for the app pilot ring, and a documented coexistence + rollback approach (SCCM keeps deploying until Intune is validated; revert by re-enabling the SCCM deployment). Why: Every app is validated on a pilot before broad cutover; cutover must be reversible. Verify: Entra → `Groups` shows the pilot ring group; the coexistence/rollback plan is written. Implement: Create the ring groups (or run `SCCMAPP-Set-AppRingTargeting.ps1`); document rollback per global requirement G7.

Checklist

- Intune licensing + Intune Administrator — `Apps` → `Windows` reachable
- Configuration Manager read access + module — `SCCMAPP-Discovery.ps1`
- Win32 Content Prep Tool present
- Source installers / content recovered per app
- Detection method captured per app (Intune-supported type)
- Dependency + supersedence map reviewed
- Packaging workstation staged
- Pilot ring group created; coexistence + rollback plan written

3. Implementation Guide — SCCM Application Migration to Intune Win32

Follow these steps **in order**. Inventory (Step 1) runs against the **Configuration Manager console** (`Software Library` → `Application Management` → `Applications`) and module. Repackaging (Step 3) runs on the packaging workstation. Everything else is in the **Microsoft Intune admin center**

(`https://intune.microsoft.com` → `Apps` → `Windows`).

> ***Model:*** *Inventory every SCCM app and its constructs, repackage each into* `.intunewin` *with detection/requirement/dependency parity, assign to a pilot ring, validate install + detection, then expand to broad and retire the SCCM deployment. The pilot ring is the gate — a bad repackage never reaches production, and SCCM keeps deploying (coexistence) until Intune is proven.*

Named objects (reuse these names verbatim everywhere)

We use the `<Purpose> – <Scope> – <Stage>` convention so every object is self-describing and searchable. For the worked example app `LOB Reader` (SCCM application `LOB Reader`, MSI deployment type, installer `LOBReader.msi`):

Object	Name to create
Pilot ring group	`App Migration – Pilot Devices – Ring 1`
Broad ring group	`App Migration – Broad Devices – Ring 2`
Migrated Win32 app	`LOB Reader – Win32 – Migrated`
Dependency Win32 app	`VC++ Redistributable x64 – Win32 – Dependency`
Superseded prior app	`LOB Reader – Win32 – Prior Version`
Detection rule identifier	`LOB Reader – Detection – MSI ProductCode`
Requirement rule	`LOB Reader – Requirement – x64 MinOS22H2`

Step 1 — Inventory the SCCM application estate

1. In the Configuration Manager console: `Software Library` → `Application Management` → `Applications`. Review the in-scope applications.

1. Run `SCCMAPP-Discovery.ps1` (uses `Get-CMApplication` / `Get-CMDeploymentType`) to export, per

app: deployment types, detection methods, install/uninstall commands, requirements, dependencies, and supersedence, plus the content source path.

1. Review the export and confirm each app has a captured, Intune-supported detection method

(Requirement 2.5) and a known silent install command (Requirement 2.4).

Step 2 — Prioritize into migration waves

1. Sort the inventory into waves: start with simple MSI apps, high value, low dependency depth.

2. Defer App-V, script-heavy, or user-context deployment types to a packaging-decision wave — these

may need rework rather than a straight repackage (see Overview limitations).

1. Record the wave order; the pilot ring migrates first within each wave.

Step 3 — Repackage the app into a `.intunewin`

1. On the packaging workstation, confirm the Win32 Content Prep Tool is present (Requirement 2.3): `IntuneWinAppUtil.exe` in `C:\Tools\IntuneWinAppUtil\`.
1. Stage the clean source folder from the SCCM content source, e.g. `C:\Packaging\LOBReader\` with `LOBReader.msi` inside.
1. Run `IntuneWinAppUtil.exe` (interactive, or scripted) pointing at the source folder and the setup file, output to `C:\Packaging\Output\`.
1. Confirm `C:\Packaging\Output\LOBReader.intunewin` is produced. This is the file you upload.

Step 4 — Create the ring groups

1. Entra → `Groups` → `New group` → `Security` → name `App Migration – Pilot Devices – Ring 1` (a few representative devices that today receive the SCCM deployment). `Create`.
1. Create `App Migration – Broad Devices – Ring 2` (the rest of the target scope). `Create`.

(Or run `SCCMAPP-Set-AppRingTargeting.ps1`.)

Step 5 — (If needed) Repackage and create the dependency app first

If the SCCM app declared a dependency, repackage and create that app **before** the main app.

1. Wrap the runtime (Step 3) and, in Intune → `Apps` → `Windows` → `Add`, create a Win32 app named `VC++ Redistributable x64 – Win32 – Dependency`.
1. Give it its own correct install/uninstall commands and a detection rule (e.g. its registry key).
2. You do **not** assign the dependency separately — it can auto-install via the main app's dependency declaration in Step 9.

Step 6 — Create the migrated Win32 app

1. Intune → `Apps` → `Windows` → `Add`. For `App type` choose `Windows app (Win32)`. `Select`.
2. `App package file`: browse to `C:\Packaging\Output\LOBReader.intunewin`. `OK`.
3. `App information`: `Name` = `LOB Reader – Win32 – Migrated`; set `Publisher` and a short `Description`. `Next`.

Step 7 — Set install/uninstall commands and behavior (carry over from SCCM)

On the `Program` tab, reuse the SCCM deployment type's programs (from the Step 1 export):

1. `Install command`: e.g. `msiexec /i "LOBReader.msi" /qn /norestart`.
2. `Uninstall command`: e.g. `msiexec /x {PRODUCT-GUID} /qn /norestart`.
3. `Install behavior`: `System` (per-device; correct for most SCCM-deployed line-of-business apps).
4. `Device restart behavior`: `No specific action` (the `/norestart` switch handles reboots).
5. `Return codes`: keep the defaults (`0`, `1707` success; `3010` soft reboot; `1641` hard reboot;

`1618` retry) unless the vendor documents others. `Next`.

Step 8 — Author requirement rules to match SCCM

On the `Requirements` tab, mirror the SCCM requirement rules from the Step 1 export:

1. `Operating system architecture`: e.g. `64-bit`.
2. `Minimum operating system`: your floor, e.g. `Windows 11 22H2`.
3. Add any custom requirement only if SCCM had one, named

`LOB Reader – Requirement – x64 MinOS22H2` (a file, registry, or script rule). `Next`.

Step 9 — Author detection rules to match SCCM (parity)

On the `Detection rules` tab, choose `Manually configure detection rules` (more reliable than auto), then Add the rule captured in Requirement 2.5, identified as `LOB Reader – Detection – MSI ProductCode`:

- `Rule type: MSI` → enter the SCCM detection method's `MSI product code` `{PRODUCT-GUID}` (best for

MSIs); optionally check the product version.

- *Alternative (EXE apps)* — `Rule type: File` → path `C:\Program Files\LOB\Reader\`, file

`LOBReader.exe`, `Detection method: version`, operator `greater than or equal to`, value the SCCM version.

- *Alternative* — `Rule type: Registry` → the app's uninstall key + a `DisplayVersion` check.
- *Alternative (script detection in SCCM)* — `Rule type: Use a custom detection script` → upload the

ported detection script.

Use **exactly one** authoritative detection rule type per app — the same logic SCCM used. `Next`.

Step 10 — Declare dependencies and supersedence to match SCCM

1. `Dependencies` tab → `Add` → select `VC++ Redistributable x64 – Win32 – Dependency` → set

`Automatically install` = `Yes` so it installs first if missing (matches the SCCM dependency). `Next`.

1. `Supersedence` tab (when the SCCM app superseded an older one) → `Add` → select the prior app

`LOB Reader – Win32 – Prior Version` → choose `Uninstall previous version = Yes` (clean replace) or `No` (in-place upgrade), matching the SCCM supersedence behavior. `Next`.

Step 11 — Assign to the pilot ring (start coexistence)

1. `Assignments` tab → under `Required` → `Add group` → select

`App Migration – Pilot Devices – Ring 1`. (Use `Available for enrolled devices` instead if the SCCM deployment was Available/optional via Software Center.)

1. Do **not** assign the broad ring yet — the pilot is the gate. `Next` → `Review + create` →

`Create`.

1. **Coexistence:** leave the SCCM deployment in place for these devices for now — SCCM continues to

deploy until the Intune app is validated on the same pilot devices.

Step 12 — Apply and validate on the pilot (per-app gate)

1. On a pilot device, sync policy (`Settings` → `Accounts` → `Access work or school` → `Info` →

`Sync`).

1. Run `SCCMAPP-Validation.ps1` to confirm the app installed and the Intune detection rule matches

what SCCM detected (install/detection parity). Complete the **Testing Guide**.

1. Run `SCCMAPP-Report.ps1` to confirm Intune reports `Installed` for the pilot devices.
2. **Gate:** the app passes only when install + detection parity holds on the pilot. If it fails,

fix the package and re-validate; the SCCM deployment (still live) means users never lose the app.

Step 13 — Retire the SCCM deployment for the pilot, then expand

1. Once the pilot passes, in the Configuration Manager console remove/disable the SCCM `Deployment`

for that app to the pilot devices (coexistence → cutover for Ring 1).

1. In Intune, edit the app's `Assignments` → under `Required` → `Add group` →

`App Migration – Broad Devices – Ring 2`. `Save`.

1. Validate the broad ring, then retire the SCCM deployment for the broad scope.
2. Re-run `SCCMAPP-Report.ps1` each cycle for repackage coverage + deployment success, and chase any

`Failed` devices with `SCCMAPP-Remediation.ps1` (default `-WhatIf`).

Order of operations

`Inventory SCCM apps → prioritize waves → repackage .intunewin → ring groups → dependency app → migrated app → install/uninstall commands → requirements → detection → dependencies + supersedence → assign pilot (coexistence) →`

`validate parity → retire SCCM (pilot) + expand broad → retire SCCM (broad)`

5. Testing Guide — SCCM Application Migration to Intune Win32

Run per app after the pilot assignment applies (Implementation Step 12) and before retiring the SCCM deployment. Each app must clear the pilot gate before broad cutover.

Test 1 — Repackage produced a valid `.intunewin`

Do: Confirm `C:\Packaging\Output\<app>.intunewin` exists and uploaded without error in Intune. Expected: The Win32 app was created from the `.intunewin` with the migrated name.

Test 2 — Install/uninstall parity

Do: On a pilot device, confirm the app installed via the Intune command; run `SCCMAPP-Validation.ps1`. Expected: The app installs with the same commands SCCM used; uninstall command is present.

Test 3 — Detection parity

Do: Run `SCCMAPP-Validation.ps1` (checks the app's detection rule — MSI product code / file / registry / script) and confirm Intune → app → `Device install status` shows `Installed`. Expected: The Intune detection rule matches the same evidence SCCM detected; status `Installed`.

Test 4 — Requirements applied

Do: Confirm the app is offered only to devices meeting the requirement rules (architecture / min OS) that mirror SCCM. Expected: Devices outside the requirements are `Not applicable`, as they were in SCCM.

Test 5 — Dependencies and supersedence

Do: On a device missing the prerequisite, confirm the dependency auto-installs first; where the app supersedes a prior version, confirm the replace/upgrade behavior matches SCCM. Expected: Dependency installs first; supersedence replaces or upgrades per the SCCM behavior.

Test 6 — Assignment (ring) parity

Do: Run `SCCMAPP-Set-AppRingTargeting.ps1` (or check `Assignments`) — pilot ring targeted, broad not yet. Expected: The same devices that received the SCCM deployment are in the pilot ring; broad deferred.

Test 7 — Coverage and deployment-success report

Do: Run `SCCMAPP-Report.ps1`. Expected: Per-app repackage coverage and Intune install-success counts; laggards flagged.

Pass/fail gate

Gate	Requirement
Per-app pilot pass	Tests 1–5 pass on the pilot; install + detection parity confirmed before retiring the SCCM deployment
Broad pass	Tests 2, 3, 6, 7 pass for the broad ring before retiring the SCCM deployment for broad scope

6. Troubleshooting Guide — SCCM Application Migration to Intune Win32

Use `SCCMAPP-Discovery.ps1` to re-read the SCCM source, `SCCMAPP-Validation.ps1` for per-device install + detection parity, `SCCMAPP-Report.ps1` for coverage + deployment success, and `SCCMAPP-Remediation.ps1` (default `-WhatIf`) for safe fixes.

A. App installs but Intune reports Failed / not detected

Symptom	Cause	Remediation
Device has the app, Intune says Failed	Detection rule does not match how the app really installs	Re-check the detection rule against the SCCM detection method (`SCCMAPP-Discovery.ps1`); fix the MSI product code / file path / registry / script to match; re-sync
Detection never true	Wrong product code or 32/64-bit path/registry view	Confirm the exact MSI product code; use the correct `Program Files` / registry hive for the app's bitness

B. Install fails on the device

Symptom	Cause	Remediation
Exit code non-zero	Wrong silent switches carried over, or missing dependency	Confirm the install command matches the SCCM deployment type; ensure the dependency app auto-installs first; run `SCCMAPP-Remediation.ps1`
Needs a runtime	Dependency not declared in Intune	Package/create the dependency app and add it under `Dependencies` with `Automatically install = Yes`

C. App offered to the wrong devices

- Requirement rules don't match SCCM → re-check architecture / minimum OS / custom rules against the

Step 1 export so `Not applicable` devices match SCCM's behavior.

- Ring targeting wrong → run `SCCMAPP-Set-AppRingTargeting.ps1`; confirm the pilot ring holds the

same devices that received the SCCM deployment.

D. Supersedence / upgrade problems

Symptom	Cause	Remediation
Old and new versions both present	Supersedence not configured or set to in-place when a clean replace was needed	Add supersedence to the prior app; set `Uninstall previous version = Yes` to match SCCM's replace behavior
Upgrade removed the app	`Uninstall previous version = Yes` when an in-place upgrade was expected	Set it to `No` for in-place upgrades per the installer

E. Both SCCM and Intune fighting over the app

- During coexistence, keep SCCM deploying but do not let both authorities enforce a conflicting state

on the same device. Retire the SCCM deployment for a ring only after the Intune app is validated there. If a conflict is seen, remove/disable the SCCM `Deployment` for those devices.

F. Rollback

- If a migrated app misbehaves on the pilot, re-enable / keep the SCCM deployment for those devices

and remove the Intune assignment for the ring — coexistence means the app never disappears. Fix the package and re-validate before retrying cutover.

G. Diagnostics quick reference

```
Get-CMApplication | Select-Object LocalizedDisplayName, NumberOfDeploymentTypes
Get-CMDeploymentType -ApplicationName 'LOB Reader'   # detection, install command, requirements
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*' |
    Select-Object DisplayName, DisplayVersion         # verify detection evidence on the device
Get-Content "$env:ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log" -Tai
```

The Intune Management Extension log (`IntuneManagementExtension.log`) records Win32 app install and detection results on the device.

H. When to escalate

If the install command matches SCCM, the dependency installs, the detection rule matches the on-disk evidence, and Intune still reports Failed after a sync and a re-evaluation, collect the Intune Management Extension log and the app's `Device install status` details and open a Microsoft support case.

7. Best Practices — SCCM Application Migration to Intune Win32

• **Inventory before you touch anything.** Export every app's deployment types, detection methods, requirements, dependencies, and supersedence from SCCM first — the estate encodes years of decisions you must not lose.

• **Migrate in waves, simple first.** Start with clean MSI apps that have shallow dependencies; defer App-V, script-heavy, and user-context deployment types to a packaging-decision wave.

• **One authoritative deployment type per app.** When an SCCM app has several deployment types, pick the one that represents the estate and repackage that; note the rest for later.

• **Detection parity is the whole game.** Carry over the exact MSI product code / file / registry / script detection SCCM used — Intune reports `Installed` only when the rule matches.

• **Use coexistence deliberately.** Leave the SCCM deployment live until the Intune app is validated on the same devices; only then retire the SCCM deployment for that ring. Nobody loses software.

• **Pilot ring is the gate.** Validate install + detection parity on a small ring before broad cutover — a bad repackage never reaches the fleet.

• **Recreate dependencies and supersedence, don't skip them.** Package dependency apps first and set auto-install; mirror supersedence so version replacement behaves as it did in SCCM.

• **Name everything with `<Purpose> – <Scope> – <Stage>`.** Reuse the same names for apps, detection rules, requirements, and ring groups so the migration is searchable and auditable.

• **Prove coverage with a report.** Track repackage coverage and deployment success with `SCCMAPP-Report.ps1` and chase `Failed` devices before decommissioning SCCM.

• **Retire SCCM last, ring by ring.** Remove the SCCM deployment only after the matching Intune ring is validated — never both authorities enforcing the same app in conflict.

• **Document the mapping.** Record each SCCM app → Intune Win32 app, its detection rule, and its ring for audit and repeatability.

8. Deliverables Summary — SCCM Application Migration to Intune Win32

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	SCCM→Intune Win32 mapping, the repackage-and-validate model, coexistence, ringed cutover, benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Migrations)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: inventory, repackage, detection/requirement/dependency parity, assign, coexistence, validate, cut over, retire SCCM
5. Testing Guide	Per-app parity validation with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`SCCMAPP-Discovery.ps1`	Inventories SCCM applications + deployment types + detection methods, requirements, dependencies, supersedence (Configuration Manager module). Read-only
`SCCMAPP-Validation.ps1`	Post-migration install + detection parity check on a device (and via Graph install status). Read-only
`SCCMAPP-Report.ps1`	Repackage coverage + Intune deployment-success report (Microsoft Graph). Read-only
`SCCMAPP-Remediation.ps1`	Safe remediation of failed installs; default `-WhatIf`
`SCCMAPP-Set-AppRingTargeting.ps1`	Builds the pilot/broad app-assignment ring groups

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the Phoenixtekk model: Assess → Configure → Pilot → Validate → Expand.

Outcome

Your SCCM application estate repackaged into Intune Win32 with detection, requirement, and assignment parity — validated per app under SCCM coexistence and cut over ring by ring — so no user loses software

during the move, and the SCCM deployment is retired only once Intune is proven.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SCCMAPP-Discovery.ps1

Purpose: Reads the Configuration Manager application estate and exports, per application: each deployment type, its install/uninstall command, detection method, requirements, dependencies, and supersedence — the source-of-truth map for repackaging into Intune Win32. HOW TO RUN Run on a workstation with the Configuration Manager console/module and a Read-only Analyst (or better) SCCM role, from the site drive: powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Discovery.ps1 -SiteCode P01 -CsvPath C:\Temp\sccm-apps.csv EXPECTED RESULTS A per-deployment-type table (app, deployment type, install command, detection type, dependency + supersedence flags) on screen and, if -CsvPath is given, a CSV to migrate from. .NOTES Phoenixtekk Workplace Marketplace — Migrations. Read-only against Configuration Manager. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$SiteCode, [string]$CsvPath)

How to run: Run on a workstation with the Configuration Manager console/module and a Read-only Analyst (or better) SCCM role, from the site drive: powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Discovery.ps1 -SiteCode P01 -CsvPath C:\Temp\sccm-apps.csv EXPECTED RESULTS A per-deployment-type table (app, deployment type, install command, detection type, dependency + supersedence flags) on screen and, if -CsvPath is given, a CSV to migrate from. .NOTES Phoenixtekk Workplace Marketplace — Migrations. Read-only against Configuration Manager. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$SiteCode, [string]$CsvPath)

SCCMAPP-Remediation.ps1

Purpose: For a device where the migrated app reports Failed: triggers an Intune/MDM sync so the Win32 app re-evaluates, and re-runs the Intune Management Extension detection cycle. Reports what it would do; makes changes only when -WhatIf is turned off with -Execute. HOW TO RUN Elevated, on the affected device. Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Remediation.ps1 Apply the sync/re-evaluate: powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Remediation.ps1 -Execute EXPECTED RESULTS In preview: the actions that would run. With -Execute: an MDM sync requested and the IME service restarted so the app re-evaluates. Re-run SCCMAPP-Validation.ps1 after a policy cycle. .NOTES Phoenixtekk Workplace Marketplace — Migrations. Default is preview (-WhatIf); -Execute to apply. #> [CmdletBinding()] param([switch]$Execute) $whatIf = -not $Execute Write-Host "=== SCCMAPP Remediation — $($env:COMPUTERNAME) ($(if($whatIf){'PREVIEW / -WhatIf'}else{'EXECUTE'})) ===" -ForegroundColor Cyan

How to run: Elevated, on the affected device. Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Remediation.ps1 Apply the sync/re-evaluate: powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Remediation.ps1 -Execute EXPECTED RESULTS In preview: the actions that would run. With -Execute: an MDM sync requested and the IME service restarted so the app re-evaluates. Re-run SCCMAPP-Validation.ps1 after a policy cycle. .NOTES Phoenixtekk Workplace Marketplace — Migrations. Default is preview (-WhatIf); -Execute to apply. #> [CmdletBinding()] param([switch]$Execute) $whatIf = -not $Execute Write-Host "=== SCCMAPP Remediation — $($env:COMPUTERNAME) ($(if($whatIf){'PREVIEW / -WhatIf'}else{'EXECUTE'})) ===" -ForegroundColor Cyan

SCCMAPP-Report.ps1

Purpose: Lists the Intune Win32 (win32LobApp) apps and, for each, pulls the deployment install summary (installed / failed / pending / not applicable) so you can see repackage coverage and how the migrated apps are actually deploying across the fleet. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Report.ps1 -CsvPath C:\Temp\sccmapp-report.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A per-app table of installed/failed/pending counts and a total Win32-app coverage line; an optional per-app CSV. .NOTES Phoenixtekk Workplace Marketplace — Migrations. Read-only against Graph. #> [CmdletBinding()] param([string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementApps.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Report.ps1 -CsvPath C:\Temp\sccmapp-report.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A per-app table of installed/failed/pending counts and a total Win32-app coverage line; an optional per-app CSV. .NOTES Phoenixtekk Workplace Marketplace — Migrations. Read-only against Graph. #> [CmdletBinding()] param([string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementApps.Read.All' -NoWelcome

SCCMAPP-Set-AppRingTargeting.ps1

Purpose: Creates (idempotently) the Entra security groups used to assign migrated Win32 apps by ring, following the - - convention: "App Migration - Pilot Devices - Ring 1" and "App Migration - Broad Devices - Ring 2". Optionally adds device object IDs to the pilot ring so the same devices that received the SCCM deployment get the migrated app first. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Set-AppRingTargeting.ps1 powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Set-AppRingTargeting.ps1 -PilotDeviceIds id1,id2 Requires the Microsoft Graph SDK and Group.ReadWrite.All (+ Device.Read.All to add members). EXPECTED RESULTS Both ring groups exist (created if missing) and their object IDs are printed; any supplied pilot device IDs are added to the pilot ring. .NOTES Phoenixtekk Workplace Marketplace — Migrations. Creates/updates Entra groups. #> [CmdletBinding()] param([string[]]$PilotDeviceIds) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','Device.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Set-AppRingTargeting.ps1 powershell -ExecutionPolicy Bypass -File .\SCCMAPP-Set-AppRingTargeting.ps1 -PilotDeviceIds id1,id2 Requires the Microsoft Graph SDK and Group.ReadWrite.All (+ Device.Read.All to add members). EXPECTED RESULTS Both ring groups exist (created if missing) and their object IDs are printed; any supplied pilot device IDs are added to the pilot ring. .NOTES Phoenixtekk Workplace Marketplace — Migrations. Creates/updates Entra groups. #> [CmdletBinding()] param([string[]]$PilotDeviceIds) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','Device.Read.All' -NoWelcome

SCCMAPP-Validation.ps1